

PYTHON FOR QUANT TRADERS · PART I

Python Basics — เริ่มเขียนโค้ด บรรทัดแรก

บทที่ 6–10: Setup · Variables · Control Flow · Data Structures ·
NumPy/Pandas · Visualization

จบ Part นี้ → โหลดข้อมูลตลาด วิเคราะห์ และ plot กราฟได้จริง

สิ่งที่ทำได้เมื่อจบ **PART I**

- โหลดข้อมูลราคาหุ้น/crypto จาก CSV เข้า Python
- คำนวณ daily return, rolling mean, volatility ได้ใน 3–5 บรรทัด
- กรองข้อมูล หาวันที่ผลตอบแทนเกิน threshold ได้
- plot กราฟราคาและ return ได้ พร้อม label และ title

ทุกอย่างนี้เป็นพื้นฐานของ **quant analysis** ทั้งหมดที่จะตามมา

บทที่ 6: Setup & Python Basics

แก่นของบทนี้

Python คือภาษาที่ "อ่านออกเหมือนภาษาอังกฤษ" — ไม่ต้องประกาศ type, ไม่ต้อง compile, รันได้ทันที บทนี้ตั้ง environment และสร้างความคุ้นเคยกับ variable, type, และ operator ซึ่งเป็นอิฐก้อนแรกของทุก program

6.1 ติดตั้ง Environment

ขั้นตอน 4 ขั้น (ทำครั้งเดียว)

1. **Python 3.10+** — ดาวน์โหลดจาก python.org เลือก "Add to PATH" ตอนติดตั้ง
2. **VS Code** — ดาวน์โหลดจาก code.visualstudio.com → ติดตั้ง Extension "Python" ของ Microsoft
3. **Terminal** — เปิด Terminal ใน VS Code (Ctrl+`) แล้วรัน: `pip install numpy pandas matplotlib`
4. **ทดสอบ** — สร้างไฟล์ `test.py` พิมพ์ `print("Hello Quant")` แล้วรัน (F5 หรือ `python test.py`)

Running Example: ตลอด Part I — วิเคราะห์ราคา BTC

ทุกบทใน Part I จะต่อยอดจากตัวอย่างเดียวกัน: โหลดราคา BTC 1 ปี → คำนวณ return → หา signal → plot กราฟ
บท 6 เริ่มจาก variable พื้นฐาน → บท 9 จะเป็น DataFrame จริง → บท 10 จะ plot ออกมาให้เห็น

6.2 Variables และ Types

ใน Python ไม่ต้องประกาศ type — Python รู้เองว่าตัวแปรเก็บอะไร

```
# กำหนดค่าตัวแปร price = 65000 # int (จำนวนเต็ม) return_pct = 0.032 # float (ทศนิยม) symbol = "BTCUSDT" # str (ข้อความ) is_long = True # bool (True/False)
```

```
# ดู type print(type(price)) # <class 'int'> print(type(return_pct))# <class 'float'>
```

Type	ตัวอย่าง	ใช้กับ
int	65000 , -3 , 0	จำนวนหุ้น, วัน, index
float	0.032 , 65000.5	ราคา, ผลตอบแทน, volatility
str	"AAPL" , "2024-01-01"	ชื่อหุ้น, วันที่, label
bool	True , False	signal on/off, position open/close

ชื่อตัวแปรที่ดีใน Quant Code

- ใช้ snake_case: `daily_return` ไม่ใช่ `dailyReturn`
- ชื่อบอกความหมาย: `btc_price_usd` ดีกว่า `p`
- หลีกเลี่ยง `l` , `0` , `I` (สับสนกับ 1, 0)
- ค่า constant ใช้ตัวใหญ่: `RISK_FREE_RATE = 0.05`

6.3 อ่านโค้ดให้ออกเสียง — ทักษะแรกก่อนหัดเขียน

คนอ่านหนังสือออกเพราะรู้ว่าตัวอักษรแต่ละตัว "ออกเสียง" อย่างไร โค้ดก็เหมือนกัน — **ทุกสัญลักษณ์มีเสียงอ่านในหัว** ถ้ารู้เสียงอ่าน บรรทัดโค้ดจะกลายเป็นประโยคภาษาไทยธรรมดา และนี่คือทักษะที่สำคัญที่สุดในยุค AI: ต่อให้ไม่เคยเขียนเอง คุณก็อ่านโค้ดที่ **AI** เขียนให้ออกได้ เหมือนเด็กที่อ่านหนังสือออกก่อนจะเขียนเรียงความเป็น

```
a = "hello world"
```

🔗**อ่านว่า:** เอา "hello world" มาใส่ใน a — เครื่องหมาย = ทำงานจากขวาไปซ้ายเสมอ: ค่าทางขวา ถูกใส่เข้า "กล่อง" ทางซ้าย

```
price = price + 10
```

🔗**อ่านว่า:** เอาค่า price เดิม บวก 10 แล้วใส่กลับเข้า price — บรรทัดนี้**ไม่ใช่**สมการคณิตศาสตร์ (ในคณิต $price = price + 10$ เป็นไปไม่ได้!) แต่คือคำสั่ง "อัปเดตค่า"

ตารางเสียงอ่าน — ใช้ตลอดทั้งเล่ม

สัญลักษณ์	อ่านว่า	ตัวอย่าง	อ่านทั้งบรรทัดว่า
<code>=</code>	"เอา...มาใส่ใน..."	<code>z = 2.5</code>	เอา 2.5 มาใส่ใน z
<code>==</code>	"ถามว่า...เท่ากับ...ไหม"	<code>z == 2.5</code>	ถามว่า z เท่ากับ 2.5 ไหม (ได้คำตอบ True/False)
<code>!=</code>	"ถามว่า...ไม่เท่ากับ...ไหม"	<code>signal != 0</code>	ถามว่า signal ไม่เท่ากับ 0 ใช่ไหม
<code>>=</code>	"ถามว่า...อย่างน้อย...ไหม"	<code>z >= 2.0</code>	ถามว่า z มากกว่าหรือเท่ากับ 2.0 ไหม
<code>.</code>	"ของ" (อ่านย่อมาจากขวา)	<code>df.mean()</code>	ค่าเฉลี่ยของ df
<code>[]</code>	"หยิบตัวที่..."	<code>prices[0]</code>	หยิบตัวแรกจาก prices
<code>()</code> หลังชื่อ	"สั่งให้ทำงาน"	<code>calc_return()</code>	สั่งให้ calc_return ทำงาน
<code>:</code> ท้ายบรรทัด	"แล้วทำต่อไปนี้"	<code>if z > 2:</code>	ถ้า z มากกว่า 2 แล้วทำต่อไปนี้

⚠ กับดักอันดับ 1 ของมือใหม่: `=` กับ `==`

```
if signal = 1: # SyntaxError! ใช้ "ใส่ค่า" ในที่ที่ต้อง "ถาม" if signal == 1:
# ✓ "ถามว่า signal เท่ากับ 1 ไหม"
```

จำง่าย ๆ: หนึ่งขีด = สั่ง (ใส่ค่า), สองขีด = ถาม (เปรียบเทียบ) — โชคดีที่ Python พ้อง error ทันทีถ้าใช้ผิดใน if แต่ภาษาอื่นบางภาษาปล่อยผ่าน แล้ว bug จะตามหลอกหลอน

วิธีฝึกอ่านโค้ด

ทุกครั้งที่เจอโค้ดในเล่มนี้ ลองอ่านออกเสียงเป็นภาษาไทยทีละบรรทัดก่อนดูคำอธิบาย ถ้าอ่านแล้วเป็นประโยคที่เข้าใจได้ แปลว่าคุณเข้าใจโค้ดบรรทัดนั้นแล้ว

จริง ๆ — จากบทนี้ไป จะมีกล่อง 📦 "อ่านว่า" แทรกใต้โค้ดสำคัญ เพื่อให้เทียบเสียงอ่านของตัวเองได้

6.4 Operators และ Expressions

```
# Arithmetic operators price_today = 65000 price_yesterday = 63000
simple_return = (price_today - price_yesterday) / price_yesterday
print(f"Return: {simple_return:.2%}") # Return: 3.17% # Operators ที่ใช้บ่อย
profit = 1000 tax_rate = 0.15 net = profit * (1 - tax_rate) # = 850.0 days
= 252 annual_vol = 0.03 * (days ** 0.5) # ** = ยกกำลัง (ทำไมไม่ใช้ ^ ดูกล่องด้านล่าง)
shares = 1000 remainder = shares % 100 # % = เศษจากหาร: 1000 % 100 = 0 lots =
shares // 100 # // = หารปัดทิ้งทศนิยม: 1000 // 100 = 10
```

คำถามที่พบบ่อย: ทำไม `\*\*` ไม่ใช้ `^` แบบคณิต?

ใน Python เครื่องหมาย `^` ถูกสงวนไว้สำหรับ "XOR" (bitwise operation ที่ใช้ใน cryptography) จึงต้องใช้ `**` แทนการยกกำลัง — ไม่ผิดปกติ แค่ convention ของภาษา

สำหรับ `//` และ `%`: ถ้า `1000 / 100 = 10.0` (float) แล้ว `1000 // 100 = 10` (int, ปัดทิ้งทศนิยม) และ `1000 % 100 = 0` (เศษ) — สามสิ่งนี้เป็นชุดเดียวกัน: ผลหาร, ผลหารปัดลง, และเศษที่เหลือ

`range(1, 5)` ให้ค่า 1, 2, 3, 4 (ไม่รวมตัวสุดท้าย 5) — rule เดียวกับ slicing

`[1:5]` นี่เป็น convention ของ Python เพื่อให้

`len(range(1, 5)) == 5 - 1 == 4` นับออกง่าย

6.5 f-string — แสดงผลแบบมีรูปแบบ

f-string คือวิธี "ประกอบข้อความ" สำหรับแสดงผล — ในเล่มนี้เราใช้ **f-string** เฉพาะใน `print()` ตอนแสดงผลลัพธ์สุดท้ายเท่านั้น การคำนวณจริงจะแยกเป็นตัวแปรชื่อชัด ๆ เสมอเพื่อให้อ่านง่าย (AI ชอบยัดการคำนวณเข้าไปใน f-string เลย — เดี่ยวบทหลังจะฝึกอ่านแบบนี้)

```
symbol = "BTC" price = 65432.789 ret = 0.0317 # f-string: ใส่ตัวแปรใน { } ได้เลย
print(f"{symbol} ราคา: ${price:,.2f}") # BTC ราคา: $65,432.79
print(f"Return: {ret:.2%}") # Return: 3.17%
print(f"Annualized: {ret * 252:.1f}%") # Annualized: 8.0%
```

Format codes ที่ใช้บ่อยใน Quant

Code	ความหมาย	ตัวอย่าง	ผลลัพธ์
<code>:.2f</code>	ทศนิยม 2 ตำแหน่ง	<code>f"{65432.789:.2f}"</code>	65432.79
<code>:.0f</code>	ไม่มีทศนิยม	<code>f"{65432:.0f}"</code>	65432
<code>:,</code>	คั่นหลักพัน	<code>f"{65432:,}"</code>	65,432
<code>:.2%</code>	แปลงเป็น % ทศนิยม 2	<code>f"{0.0317:.2%}"</code>	3.17%
<code>:.4f</code>	ทศนิยม 4 ตำแหน่ง	<code>f"{0.0317:.4f}"</code>	0.0317

ตัวอย่าง: คำนวณและแสดงผล Trade Summary

```
entry_price = 63000.0 exit_price = 65450.0 position_size = 0.5 # BTC
pnl = (exit_price - entry_price) * position_size ret = (exit_price -
entry_price) / entry_price print(f"Entry: ${entry_price:,.2f}")
print(f"Exit: ${exit_price:,.2f}") print(f"P&L: ${pnl:,.2f}")
print(f"Return: {ret:.2%}")
```

► OUTPUT Entry: \$63,000.00 Exit: \$65,450.00 P&L: \$1,225.00 Return: 3.89%

► แบบฝึกหัด: คำนวณ Annualized Return

ถ้า daily return = 0.1% (0.001) คำนวณ simple annualized return (252 วัน) และแสดงผลแบบ %

```
daily_ret = 0.001 annual_ret = daily_ret * 252 # simple annualization
print(f"Annualized Return: {annual_ret:.1%}") # Annualized Return:
25.2% # หรือแบบ compound: annual_compound = (1 + daily_ret) ** 252 - 1
print(f"Compound Annual: {annual_compound:.1%}") # Compound Annual:
28.4%
```

บทที่ 7: Control Flow & Functions

แก่นของบทนี้

Control flow คือการบอก program ว่า "ทำอะไรเมื่อเงื่อนไขแบบไหน" และ "ทำซ้ำกี่ครั้ง" — ใน Quant ใช้สร้าง signal logic, loop ผ่านข้อมูล, และ function ที่ reuse ได้ทั้ง codebase

7.1 เงื่อนไขคืออะไร — คำถามที่ตอบได้แค่ True/False

ก่อนรู้จัก if ต้องเข้าใจก่อนว่า "เงื่อนไข" คืออะไร: เงื่อนไขคือคำถามที่คำตอบมีแค่ 2 แบบ — **True (จริง)** หรือ **False (เท็จ)**

```
z = 2.3 z > 2.0 # คำถาม: z มากกว่า 2.0 ไหม? → คำตอบ: True z == 2.0 # คำถาม: z  
เท่ากับ 2.0 ไหม? → คำตอบ: False z < -2.0 # คำถาม: z น้อยกว่า -2.0 ไหม? → คำตอบ:  
False
```

คำตอบ True/False เป็น "ค่า" จริง ๆ ใน Python — เอามาใส่ตัวแปรได้เหมือนตัวเลข:

```
is_overbought = z > 2.0 # เอาคำตอบ (True) มาใส่ใน is_overbought  
print(is_overbought) # True
```

📌**อ่านว่า:** ถามว่า z มากกว่า 2.0 ไหม แล้วเอาคำตอบมาใส่ใน is_overbought — สังเกตว่า = (ใส่ค่า) กับ > (ถาม) ทำงานคนละหน้าที่ในบรรทัดเดียวกัน: ผังขวาถามก่อน ได้คำตอบแล้วค่อยใส่

Comparison Operators — เครื่องมือสร้างคำถาม

Operator	อ่านว่า	ตัวอย่าง
<code>==</code>	เปรียบเทียบกับ...เท่ากับ... (ได้ True/False)	<code>signal == "LONG"</code>
<code>!=</code>	ไม่เท่ากับ...ใช่ไหม	<code>position != 0</code>
<code>></code> , <code><</code>	มากกว่า/น้อยกว่า...ไหม	<code>z > 2.0</code>
<code>>=</code> , <code><=</code>	อย่างน้อย / ไม่เกิน...ไหม	<code>drawdown <= -0.1</code>

7.2 if / elif / else — เครื่องจักรถามคำถามตามลำดับ

if/elif/else คือเครื่องจักรที่ถามคำถามจากบนลงล่างทีละข้อ — เจอข้อไหนตอบ "ใช่" ทำงานของข้อนั้นแล้วออกจากเครื่องทันที ข้อที่เหลือไม่ถูกถามอีกเลย

```
z = 2.3 if z > 2.0: # ถามข้อ 1: z มากกว่า 2.0 ไหม? signal = "SHORT" # ใช่ → ทำ  
บรรทัดนี้แล้วออกเลย ไม่ถามต่อ elif z < -2.0: # ถามข้อ 2 (เฉพาะเมื่อข้อ 1 ตอบ "ไม่") signal =  
"LONG" # ใช่ → ทำบรรทัดนี้แล้วออก elif abs(z) < 0.5: # ถามข้อ 3 (เฉพาะเมื่อข้อ 1 และ 2 ตอบ  
"ไม่") signal = "EXIT" # ใช่ → ทำบรรทัดนี้แล้วออก else: # ทุกข้อข้างบนตอบ "ไม่" ทั้งหมด  
signal = "HOLD" # ทำบรรทัดนี้ (else ไม่มีคำถาม = "ที่เหลือทั้งหมด") print(signal)
```

► OUTPUT SHORT

🔴**อ่านว่า:** ถ้า z มากกว่า 2.0 แล้วเอา "SHORT" ใส่ใน signal — ไม่งั้นถ้า z น้อยกว่า -2.0 แล้วเอา "LONG" ใส่ใน signal — ไม่งั้นถ้าขนาดของ z น้อยกว่า 0.5 แล้วเอา "EXIT" ใส่ — ไม่งั้น (ที่เหลือทั้งหมด) เอา "HOLD" ใส่

เส้นทางของ `z = 2.3` ในเครื่องจักรนี้:

ข้อ 1: `z > 2.0` ? (`2.3 > 2.0`)

| ตอบ "ใช่" ↓

`signal = "SHORT"` → ออกจากเครื่องทันที

ข้อ 2, ข้อ 3, else ไม่ถูกถามเลย แม้ว่า `abs(2.3) < 0.5` จะเป็น False อยู่แล้วก็ตาม

⚠ elif ไม่เท่ากับ if สองอัน — ตัวอย่างที่ผลต่างกันจริง

```
z = 3.0 # — แบบ elif: ถามต่อเมื่อข้อบนตอบ "ไม่" — if z > 2.0: action =
"SHORT" # เข้าข้อนี้ แล้วออกเลย elif z > 1.0: action = "WARN" # ไม่ถูกถาม # ผล:
action = "SHORT" ✓ # — แบบ if สองอัน: ถามทุกข้อเสมอ — if z > 2.0: action =
"SHORT" # เข้าข้อนี้ if z > 1.0: action = "WARN" # ถูกถามอีก! 3.0 > 1.0 จริง →
ทับค่าเดิม # ผล: action = "WARN" SHORT หายไปเลย
```

ถ้าเงื่อนไขเป็น "ทางเลือกที่เกิดขึ้นได้ทีละอย่าง" (signal มีได้ค่าเดียว) ต้องใช้ elif เสมอ — bug ชนิดนี้ Python ไม่ฟ้อง error โปรแกรมรันได้ปกติแต่ให้ signal ผิด ซึ่งใน trading คือเสียเงินจริง

ลำดับเงื่อนไขสำคัญ — เช็คข้อแคบก่อนข้อกว้าง

```
# ผิด: ข้อกว้าง (z > 1) อยู่บน → "กิน" ทุกกรณี z > 2 ไปด้วย if z > 1.0: level =
"WARN" # z = 3.0 ก็เข้าข้อนี้! ไม่มีวันถึง EXTREME elif z > 2.0: level =
"EXTREME" # โทษตายซาก — ไม่มีทางถูกเรียก # ✓ ถูก: ข้อแคบ/รุนแรงกว่า ขึ้นก่อน if z > 2.0:
level = "EXTREME" elif z > 1.0: level = "WARN"
```

เชื่อมหลายเงื่อนไข: and / or / not

ตัวเชื่อม	อ่านว่า	ผลเป็น True เมื่อ
and	"และ"	จริงทั้งคู่
or	"หรือ"	จริงอย่างน้อยหนึ่งข้อ
not	"ไม่/กลับด้าน"	ของเดิมเป็น False

```
# เข้าtrade เมื่อ: z สูงพอ "และ" ยังไม่มี position if z > 2.0 and position == 0:
position = -1 # ออกจากtrade เมื่อ: z กลับใกล้ 0 "หรือ" ขาดทุนเกิน limit if abs(z) <
0.5 or loss > max_loss: position = 0
```

🔴**อ่านว่า:** ถ้า z มากกว่า 2.0 และ position เท่ากับ 0 (ทั้งสองข้อต้องจริง) แล้วเอา -1 ใส่ใน position

⚠ กับดักการเขียนเงื่อนไข 2 จุด

```
# 1) เงื่อนไขช่วง — ต้องเขียนตัวแปรซ้ำทั้งสองฝั่ง if z > 1.0 and < 3.0: #
SyntaxError if z > 1.0 and z < 3.0: # ✓ if 1.0 < z < 3.0: # ✓ Python
ยอมให้เขียนแบบคณิตได้ด้วย # 2) indent (ย่อหน้า) กำหนดว่าบรรทัดไหนอยู่ "ใน" if if z >
2.0: signal = -1 log_trade() # อยู่ใน if — ทำเฉพาะเมื่อ z > 2.0 if z > 2.0:
signal = -1 log_trade() # อยู่นอก if — ทำทุกครั้ง! ต่างแค่ย่อหน้าเดียว
```

7.3 for loop — วนซ้ำ

for loop คือคำสั่ง "หยิบของจากรายการมาทีละตัว แล้วทำชุดคำสั่งเดิมซ้ำกับทุกตัว"

```
prices = [63000, 65000, 64500] for p in prices: # หยิบสมาชิกจาก prices มาใส่ p ทีละ
ตัว print(p) # ทำบรรทัดนี้กับ p ทุกรอบ
```

📌 **อ่านว่า:** สำหรับแต่ละตัวใน prices — เอาตัวนั้นมาใส่ใน p แล้วทำต่อไปนี้: print ค่า p

ดูการทำงานทีละรอบ — เทคนิคสำคัญ: เวลา loop ให้ "เดินเครื่องด้วยมือ" ตามตาราง —
จดทั้ง ค่าตัวแปร loop (i) และ state ของ list ที่กำลังสะสม (returns):

รอบที่	i	prices[i]	returns (list สะสม)
1	1	65000	[0.0317]
2	2	64500	[0.0317, -0.0077]
3	3	66000	[0.0317, -0.0077, 0.0233]
4	4	65800	[0.0317, -0.0077, 0.0233, -0.0030] → loop จบ

ตัวอย่างจริง: คำนวณ return รายวัน — ต้องใช้ "ราคาวันนี้" คู่กับ "ราคาเมื่อวาน" จึง loop
ด้วยตำแหน่ง (index) แทน:

```
prices = [63000, 65000, 64500, 66000, 65800] returns = [] # เตรียม list เปล่าไว้
เก็บผล for i in range(1, len(prices)): # i ไล่จาก 1, 2, 3, 4 (เริ่มวันที่สอง) today =
prices[i] # หยิบราคาวันนี้ yesterday = prices[i-1] # หยิบราคาเมื่อวาน (ตัวก่อนหน้า) r =
(today - yesterday) / yesterday # สูตร simple return returns.append(r) # เก็บผล
ต่อท้าย list print(returns) # [0.0317, -0.0077, 0.0233, -0.0030]
```

```
# enumerate: ได้ทั้งตำแหน่งและค่าพร้อมกัน for i, price in enumerate(prices):
print(f"วันที่ {i+1}: ${price:,}") # zip: loop สองรายการพร้อมกัน (จับคู่ตัวต่อตัว) symbols
```

```
= ["BTC", "ETH", "SOL"] weights = [0.5, 0.3, 0.2] for sym, w in zip(symbols, weights): print(f"{sym}: {w:.0%}")
```

while loop — วนจนกว่าเงื่อนไขจะเป็นเท็จ

for loop กับ while loop แตกต่างกันที่ "รู้ล่วงหน้าไหมว่าจะวนกี่รอบ":

	for loop	while loop
ใช้เมื่อ	รู้จำนวนรอบล่วงหน้า	ไม่รู้จำนวนรอบ — วนจนเงื่อนไขเปลี่ยน
หยุดเมื่อ	หมด list / range	เงื่อนไขกลายเป็น False
ตัวอย่าง Quant	คำนวณ return 252 วัน (รู้ว่า 252 วัน)	เทรดต่อจนทุนหมด (ไม่รู้วันที่วัน)

```
equity = 100000 # เงินเริ่มต้น 100,000 บาท max_loss = 90000 # ถ้าเงินเหลือน้อยกว่า 90,000
หยุดเทรด daily_return = 0.005 # สมมติกำไร/ขาดทุนวันละ 0.5% (จริงๆ จะได้จาก strategy)
while equity > max_loss: # ก่อนทุกรอบ ถาม: equity ยังมากกว่า max_loss ไหม? profit =
equity * daily_return # คำนวณกำไร/ขาดทุนวันนี้ (ใช้ค่าสุ่มแทนจริงๆ) equity = equity +
profit # อัปเดตเงินทุน # ไม่ → ออกจาก loop (หยุดเทรด) print("หยุดเทรด - ขาดทุนถึง limit")
```

🔴 **อ่านว่า:** ตราบใดที่ equity ยังมากกว่า max_loss ก็ทำต่อไปนี้: เทรดหนึ่งวันแล้วอัปเดต equity — เครื่องจะกลับขึ้นไปถามคำถามเดิมซ้ำก่อนเริ่มรอบใหม่ทุกครั้ง

⚠ Infinite loop — while ที่ไม่มีวันจบ

ถ้าในตัว loop ไม่มีอะไรทำให้เงื่อนไขเปลี่ยนเป็น **False** ได้เลย โปรแกรมจะวนตลอดกาล (ค้าง) เช่นลืมหอัปเดต equity ในตัวอย่างบน — เช็คว่า "ตัวแปรในเงื่อนไขถูกเปลี่ยนค่าข้างใน loop หรือยัง" ถ้าโปรแกรมค้าง กด Ctrl+C เพื่อหยุด

7.4 List Comprehension — Loop ในบรรทัดเดียว (ฝึกอ่านโค้ดแบบ AI ครั้งแรก)

ทุกอย่างที่ทำได้ด้วย loop ธรรมดา เขียนย่อในบรรทัดเดียวได้ด้วย "list comprehension" — คุณไม่จำเป็นต้องเขียนเป็น แต่ต้องอ่านออก เพราะ AI เขียนแบบนี้ตลอด

```
prices = [63000, 65000, 64500, 66000, 65800] # แบบที่เราเขียน (อ่านออก 100%)
returns = [] for i in range(1, len(prices)): today = prices[i] yesterday =
prices[i-1] returns.append((today - yesterday) / yesterday)
```

AI เขียนแบบนี้ — ฝึกอ่านกัน

```
returns = [(prices[i]-prices[i-1])/prices[i-1] for i in range(1, len(pr
```

สูตรอ่าน list comprehension: อ่านครึ่งหลังก่อน แล้วย้อนกลับมาครึ่งแรก

1. `for i in range(1, len(prices))` — "สำหรับแต่ละ `i` ตั้งแต่ 1 ถึงท้ายรายการ" (นี่คือ loop เดิมนั่นเอง)
2. `(prices[i]-prices[i-1])/prices[i-1]` — "คำนวณค่านี้ในแต่ละรอบ" (คือ 3 บรรทัดในตัว loop ยวบเลื่อนนิพจน์เดียว)
3. `[ ... ]` ครบทั้งหมด — "เก็บผลหุ้รอบลง list" (คือ `returns = [] + .append()`)

ทั้งบรรทัดอ่านว่า: "สร้าง list ของ return โดยคำนวณจากราคาคู่วันติดกัน ทุกวันตั้งแต่วันที่สอง" — ความหมายเหมือนโค้ด 5 บรรทัดข้างบนปะะ แคย่อ

```
# comprehension + if = กรองของ loss_days = [r for r in returns if r < 0]
print(f"วันขาดทุน: {len(loss_days)} วัน")
```

📌**อ่านว่า:** สร้าง list จาก `r` แต่ละตัวใน `returns` เฉพาะตัวที่ `r` น้อยกว่า 0 — แล้วเอามาใส่ใน `loss_days`

เมื่อ AI ใช้ if ใน comprehension + ternary — 2 รูปแบบที่ต่างกัน

```
# รูปแบบที่ 1: กรอง (filter) — if อยู่หลัง for
loss_days = [r for r in returns if r < 0]
# รูปแบบที่ 2: เลือกค่า (ternary) — if อยู่หน้า for
labels = ["loss" if r < 0 else "gain" for r in returns]
```

ทั้งสองรูปแบบมี `if` แต่ตำแหน่งต่างกัน → ความหมายต่างกันสิ้นเชิง:

1. `if` หลัง `for` = กรองของ: "เอาเฉพาะ `r` ที่ `r < 0`" → list สั้นลง
2. `if...else` หน้า `for` = เลือกค่า: "ถ้า `r < 0` ให้เป็น 'loss' ไม่งั้น 'gain'" → list ยาวเท่าเดิม แค่เปลี่ยนค่า

จำง่าย: ก่อน `for` = เลือกว่าแต่ละตัวจะเป็นอะไร, หลัง `for` = กรองว่าตัวไหนเข้าหรือออก

7.5 Functions — def

```
# นิยาม function def calc_return(price_now, price_prev): """คำนวณ simple
return ระหว่างสองราคา""" return (price_now - price_prev) / price_prev # เรียกใช้ r
```

```
= calc_return(65000, 63000) print(f"Return: {r:.2%}") # Return: 3.17% #
Default arguments def annualize(daily_vol, periods=252): """แปลง daily
volatility เป็น annualized""" return daily_vol * (periods ** 0.5)
print(annualize(0.03)) # 0.4762 (252 วัน = หุ้น) print(annualize(0.03, 365)) #
0.5733 (365 วัน = crypto)
```

Docstring — เขียนทุก function ใน Quant code

บรรทัดแรกใน function (ในเครื่องหมาย `"""`) คือ docstring อธิบาย function นั้น ช่วยเมื่อ copy code ไปให้ AI ตรวจ — AI จะเข้าใจ intent ได้ถูกต้อง และช่วย debug ได้แม่นยำขึ้น

ตัวอย่าง: Z-score Function แบบ Reusable

```
def zscore(series, window=60): """ คำนวณ rolling z-score ของ series
Args: series: list หรือ numpy array ของราคา/spread window: rolling
window (default 60 วัน) Returns: list ของ z-score """ results = [] #
เตรียม list เก็บผลลัพธ์ for i in range(len(series)): # เดินผ่านข้อมูลทีละวัน if i <
window: # วันแรกๆ ข้อมูลยังไม่ครบ window results.append(None) # → ใส่
None แทน (คำนวณไม่ได้) continue # ข้ามไปวันถัดไปเลย window_data = series[i-
window:i] # หีบข้อมูล window วันล่าสุด mean = sum(window_data) / window # ค่า
เฉลี่ยของหน้าต่างนั้น variance = sum((x - mean)**2 for x in window_data) /
window # ความแปรปรวน std = variance ** 0.5 # ส่วนเบี่ยงเบนมาตรฐาน = รากของ
variance if std == 0: # กันหารด้วยศูนย์ (ราคานิ่งสนิท) results.append(0) else:
z = (series[i] - mean) / std # สูตร z-score: ห่างจากค่าเฉลี่ยกี่ std
results.append(z) return results # ทดสอบ spread = [100, 102, 98,
105, 99, 103, 110, 97, 101, 104] z = zscore(spread, window=5)
print(z[-3:]) # z-score 3 วันล่าสุด
```

AI เขียนฟังก์ชันเดียวกันนี้ใน 1 บรรทัด — พิกอ่านกัน

```
z = (s - s.rolling(60).mean()) / s.rolling(60).std()
```

(s คือ pandas Series — จะเรียนบทที่ 9) อ่านทีละชิ้นจากในออกนอก:

1. `s.rolling(60)` — "หน้าต่างเลื่อน 60 วัน ของ s" (จุด = "ของ")
2. `s.rolling(60).mean()` — "ค่าเฉลี่ย ของ หน้าต่าง 60 วัน" = ตัวแปร mean ใน ฟังก์ชันเรา
3. `s.rolling(60).std()` — "std ของหน้าต่าง 60 วัน" = ตัวแปร std ของเรา
4. `(s - mean) / std` — สูตร Z-score เดิมเป๊ะ แต่ทำกับทุกวันพร้อมกัน (ไม่ต้อง loop เอง)

โค้ด 15 บรรทัดของเรา กับบรรทัดเดียวกัน *ทำสิ่งเดียวกัน* — pandas ซ่อน loop ไว้ข้างใน (และเร็วกว่า ~100 เท่า) เราเขียนแบบยาวก่อนเพื่อให้รู้ว่า "ข้างในมันทำอะไร" พอไปบทที่ 9 จะใช้แบบสั้นได้อย่างมั่นใจ

7.6 lambda — Function ย่อ

```
# lambda: function 1 บรรทัด ไม่มีชื่อ calc_ret = lambda p_now, p_prev: (p_now -  
p_prev) / p_prev # ใช้บ่อยกับ sorted() และ map() trades = [("BTC", 500), ("ETH",  
-200), ("SOL", 300)] by_pnl = sorted(trades, key=lambda t: t[1],  
reverse=True) print(by_pnl) # [('BTC', 500), ('SOL', 300), ('ETH', -200)]
```

► แบบฝึกหัด: เขียน function คำนวณ Sharpe Ratio

```
def sharpe_ratio(returns, risk_free=0.0, periods=252): """ คำนวณ Sharpe  
Ratio ทยอย returns: list ของ daily return (float) risk_free: annual  
risk-free rate (default 0) periods: วันซื้อขายต่อปี """ n = len(returns)  
mean_r = sum(returns) / n variance = sum((r - mean_r)**2 for r in  
returns) / n daily_std = variance ** 0.5 daily_rf = risk_free /  
periods sharpe = (mean_r - daily_rf) / daily_std * (periods ** 0.5)  
return sharpe # ทดสอบ daily_returns = [0.001, -0.005, 0.008, 0.002,  
-0.003] s = sharpe_ratio(daily_returns) print(f"Sharpe: {s:.2f}")
```

บทที่ 8: Data Structures

แก่นของบทนี้

Python มี 4 data structure หลัก — **list** (รายการเรียงลำดับ), **dict** (key-value), **tuple** (ค่าคงที่), **set** (ค่าไม่ซ้ำ) แต่ละอย่างมีจุดแข็งต่างกัน รู้จักเลือกใช้ถูกตัว code จะสั้นและเร็วขึ้นมาก

8.1 list — รายการที่เรียงลำดับ

```
prices = [63000, 65000, 64500, 66000, 65800] # Indexing (เริ่มจาก 0)
print(prices[0]) # 63000 (ตัวแรก) print(prices[-1]) # 65800 (ตัวสุดท้าย - เลขติดลบ
= นับจากท้าย) # Slicing [start:stop:step] print(prices[1:4]) # [65000, 64500,
66000] print(prices[:3]) # [63000, 65000, 64500] (3 ตัวแรก) print(prices[::
2]) # [63000, 64500, 65800] (ทุก 2 ตัว) # เพิ่ม/ลบ prices.append(67000) # เพิ่มต่อ
ท้าย prices.insert(0, 62000) # แทรกที่ตำแหน่ง 0 last = prices.pop() # ดึงตัวสุดท้ายออก
# ข้อมูลพื้นฐาน print(len(prices)) # จำนวนสมาชิก print(min(prices), max(prices)) #
ต่ำสุด สูงสุด print(sum(prices) / len(prices)) # ค่าเฉลี่ย
```

8.2 dict — Key-Value Store

dict คือโครงสร้างที่ใช้มากที่สุดในการเก็บข้อมูลตลาด — key คือชื่อ/symbol, value คือข้อมูล

```
# OHLCV ของวันหนึ่ง candle = { "open": 64000.0, "high": 66500.0, "low":
63800.0, "close": 65450.0, "volume": 28500.0 } # เข้าถึงค่า
print(candle["close"]) # 65450.0 print(candle.get("vwap", None)) # None (ไม่
มี key นั้น - ไม่error) # เพิ่ม/อัปเดต candle["body"] = candle["close"] -
candle["open"] # = 1450.0 # loop ผ่าน dict for key, value in candle.items():
print(f"{key:10s}: {value:,.1f}") # Portfolio เป็น dict ของ dict portfolio =
{ "BTC": {"size": 0.5, "entry": 63000, "side": "long"}, "ETH": {"size":
2.0, "entry": 3200, "side": "long"}, "SOL": {"size": 10.0, "entry": 145,
"side": "short"}, }
```

8.3 tuple — ค่าคงที่

```
# tuple เหมือน list แต่ immutable (เปลี่ยนไม่ได้) config = (2.0, 0.5, 60) # (entry_z,
exit_z, window) entry_z, exit_z, window = config # unpacking # ใช้เป็น key ของ
dict ได้ (list ทำไม่ได้) pair_params = { ("BTC", "ETH"): {"beta": 0.85,
```



```
"half_life": 4.2}, ("BTC", "SOL"): {"beta": 1.20, "half_life": 6.8}, }  
print(pair_params[("BTC", "ETH")]["beta"]) # 0.85
```

8.4 set — ค่าไม่ซ้ำ

```
# set เก็บเฉพาะค่าที่ไม่ซ้ำ signals_day1 = {"AAPL", "MSFT", "NVDA"} signals_day2 =  
{"MSFT", "NVDA", "TSLA"} both_days = signals_day1 & signals_day2 #  
intersection = {"MSFT", "NVDA"} either_day = signals_day1 | signals_day2 #  
union only_day1 = signals_day1 - signals_day2 # difference = {"AAPL"} # ลบ  
duplicate ออกจาก list trades = ["BTC", "ETH", "BTC", "SOL", "ETH", "BTC"]  
unique_symbols = list(set(trades)) # ["BTC", "ETH", "SOL"]
```

เลือก Data Structure ไหน?

ถ้าต้องการ	ใช้	เหตุผล
เก็บ series ราคาเรียงเวลา	list	เรียงลำดับ, append ง่าย
เก็บ OHLCV ของ 1 candle	dict	access ด้วยชื่อ field ได้
ค่า config ที่ไม่เปลี่ยน	tuple	immutable, ใช้เป็น dict key ได้
รายการ symbol ที่เคยเทรด	set	ลบ duplicate อัตโนมัติ
ข้อมูลตลาดหลายวัน	pd.DataFrame	Part I บทที่ 9

ตัวอย่าง: สร้าง Return Map ของหลายเหรียญ

```
close_prices = { "BTC": [63000, 65000, 64500], "ETH": [3100, 3200,  
3150], "SOL": [140, 148, 145], } # แบบที่เราเขียน: loop ผ่านทุกเหรียญ คำนวณ  
return ล่าสุด latest_returns = {} # เตรียม dict เปล่า for sym, prices in  
close_prices.items(): # หอับ (ชื่อเหรียญ, list ราคา) ที่ละคู่ last = prices[-1]  
# ราคาล่าสุด (ตัวสุดท้าย) prev = prices[-2] # ราคาก่อนหน้า (ตัวรองสุดท้าย)  
latest_returns[sym] = (last - prev) / prev # เก็บ return โดยใช้ชื่อเหรียญเป็น  
key for sym, r in latest_returns.items(): print(f"{sym}: {r:.2%}")
```

► OUTPUT BTC: -0.77% ETH: -1.56% SOL: -2.03%

```
latest_returns = {
    sym: (prices[-1] - prices[-2]) / prices[-2]
    for sym, prices in close_prices.items()
}
```

สูตรเดียวกับ list comprehension: อ่านบรรทัด **for** ก่อน แล้วย้อนขึ้นไปดูว่าแต่ละรอบสร้างอะไร

1. `for sym, prices in close_prices.items()` — loop เดิม: หยิบ (ชื่อเหรียญ, ราคา) ทีละคู่
2. `sym: (prices[-1] - prices[-2]) / prices[-2]` — แต่ละรอบสร้างคู่ key: value = ชื่อเหรียญ : return ค่าสุด
3. `{ ... }` ปีกกาครอบ — เก็บทุกคู่ลง dict (ปีกกา = dict, วงเล็บเหลี่ยม = list)

ความหมายเหมือนโค้ด 5 บรรทัดข้างบนทุกประการ

► แบบฝึกหัด: สร้าง **Portfolio Summary**

```
positions = { "BTC": {"entry": 60000, "current": 65000, "size": 0.5},
              "ETH": {"entry": 3000, "current": 3200, "size": 2.0}, } # จำนวน P&L
แต่ละ position for sym, pos in positions.items(): pnl = (pos["current"]
- pos["entry"]) * pos["size"] ret = (pos["current"] - pos["entry"]) /
pos["entry"] print(f"{sym}: P&L=${pnl:,.0f} ({ret:.1%})")
```

► OUTPUT BTC: P&L=\$2,500 (8.3%) ETH: P&L=\$400 (6.7%)

บทที่ 9: NumPy & Pandas

แก่นของบทนี้

NumPy และ Pandas เป็น "engine" ของ Quant Python — NumPy ทำ math บน array เร็วกว่า loop ธรรมดา 100x+, Pandas จัดการ table ข้อมูล พร้อม label วันที่ ทั้งสองรวมกันทำให้วิเคราะห์ข้อมูลตลาดหลักหมื่นแถวได้ใน milliseconds

9.1 NumPy — คณิตศาสตร์บน Array

```
import numpy as np # สร้าง array prices = np.array([63000, 65000, 64500, 66000, 65800]) # Operations ทำทั้ง array พร้อมกัน (vectorized) returns = np.diff(prices) / prices[:-1] # return ทุกวัน print(returns) # [0.0317 -0.0077 0.0233 -0.0030] # Statistics print(f"Mean: {np.mean(returns):.4f}") print(f"Std: {np.std(returns):.4f}") print(f"Max: {np.max(returns):.4f}") print(f"Min: {np.min(returns):.4f}") # Annualized Vol annual_vol = np.std(returns) * np.sqrt(252) print(f"Annual Vol: {annual_vol:.1%}")
```

ทำไม NumPy เร็วกว่า Python loop?

Python loop: อ่านแต่ละ element → แปลง type → คำนวณ → เก็บ → ไปต่อ (overhead สูง)

NumPy: ส่ง array ทั้งก้อนให้ C code → คำนวณ parallel → ได้ผล (overhead ต่ำมาก)

ผลลัพธ์: array ขนาด 1,000,000 element → NumPy เร็วกว่า loop ประมาณ 100–1,000x

9.2 ทำไม "ตารางธรรมดา" ถึงไม่พอ

ก่อนตอบว่า DataFrame คืออะไร ลองดูก่อนว่าถ้าไม่มีมัน ชีวิตเป็นยังไง — เก็บราคา 2 เหรียญด้วยเครื่องมือที่เรามีจากบทที่ 8 (list ซ้อน list = "ตารางทำมือ"):

```
# ตารางธรรมดา: list ซ้อน list table = [ # [วันที่, BTC, ETH] ["2024-01-01", 63000, 3100], ["2024-01-02", 65000, 3250], ["2024-01-03", 64500, 3200], ] # ถ้าถามง่าย ๆ: "ราคา BTC วันที่ 2024-01-02 คือเท่าไร?" # → ต้องเขียน loop ค้นเอง: for row in table:
```

```
if row[0] == "2024-01-02": # ช่องที่ 0 คือวันที่ (ต้องจำเอง) print(row[1]) # ช่องที่ 1 คือ BTC (ต้องจำเอง)
```

ใช้งานได้ แต่มีปัญหา 3 ข้อที่จะก่ดเราแน่นอนเมื่อข้อมูลโตขึ้น:

ปัญหาของตารางทำมือ

1. **ต้องจำตำแหน่งเอง** — "ช่องที่ 1 คือ BTC" ไม่มีเขียนไว้ในโค้ด ถ้าสลับคอลัมน์ในไฟล์ข้อมูลวันไหน โค้ดจะหยิบเลขผิดช่องโดยไม่มี error
2. **ค้นหาต้อง loop ทุกครั้ง** — ข้อมูล 100,000 แถว อยากรู้ราคาวันเดียว ต้องไล่ดูแถว
3. **ข้อมูลหายแล้วแถวเลื่อน** — สมมติ ETH ไม่มีข้อมูลวันที่ 2 ถ้าเก็บแยกสอง list แล้วเอามาคู่กัน ตำแหน่งที่ 1 ของ BTC จะจับคู่กับ ETH ผิดวัน → คำนวณ correlation ผิดทั้งชุด แบบเจียบ ๆ

pandas ถูกสร้างมาแก้ 3 ปัญหานี้ตรง ๆ — และกุญแจของมันคือสิ่งที่เรียกว่า **index**

9.3 Index คืออะไร — ป้ายชื่อประจำแถว

list ธรรมดาเรียกข้อมูลด้วย "ตำแหน่ง": ตัวที่ 0, ตัวที่ 1, ตัวที่ 2 — เหมือนล็อกเกอร์ที่มีแต่เลขตัว

Series ของ pandas คือ list ที่ทุกค่ามีป้ายชื่อติดอยู่ — ป้ายชื่อเหล่านั้นรวมกันเรียกว่า index:

```
import pandas as pd prices = pd.Series( [63000, 65000, 64500], # ค่า (values) index=["2024-01-01", "2024-01-02", "2024-01-03"], # ป้ายชื่อ (index) name="BTC" ) print(prices)
```

► OUTPUT 2024-01-01 63000 2024-01-02 65000 2024-01-03 64500 Name: BTC, dtype: int64

สังเกตว่า pandas พิมพ์ป้ายชื่อ (ซ้าย) คู่กับค่า (ขวา) เสมอ — ที่นี้คำถาม "ราคาวันที่ 2 ม.ค.?" ตอบได้บรรทัดเดียว ไม่ต้อง loop:

```
print(prices["2024-01-02"]) # 65000 — ถามด้วยป้ายชื่อตรง ๆ print(prices.iloc[1]) # 65000 — หรือถามด้วยตำแหน่งแบบเดิมก็ยังได้ (iloc)
```

📌 **อ่านว่า:** หยิบค่าที่ป้ายชื่อ "2024-01-02" จาก prices — index ทำให้เราคุยกับข้อมูลด้วย "ภาษาของข้อมูล" (วันที่) แทนตำแหน่ง

และพลังที่สำคัญที่สุดของ index — จับคู่แถวให้อัตโนมัติ (**alignment**) ซึ่งแก้ปัญหาคือ 3 แบบถาวร:

```
# ETH ไม่มีข้อมูลวันที่ 2 (ตลาดล่ม) btc = pd.Series([63000, 65000, 64500],
index=["2024-01-01", "2024-01-02", "2024-01-03"]) eth = pd.Series([3100,
3200], index=["2024-01-01", "2024-01-03"]) # ขาดวันที่ 2! ratio = btc / eth #
pandas จับคู่ด้วย "ป้ายชื่อ" ไม่ใช่ตำแหน่ง print(ratio)
```

```
► OUTPUT 2024-01-01 20.322581 2024-01-02 NaN 2024-01-03 20.156250
```

วันที่ 2 ได้ NaN (Not a Number = "ไม่มีข้อมูล") — pandas บอกตรง ๆ ว่าวันนี้คำนวณไม่ได้ แทนที่จะจับคู่ผิดแถวเจียบ ๆ แบบ list ถ้าเป็นตารางทำมือ ราคา ETH วันที่ 3 จะถูกจับคู่กับ BTC วันที่ 2 — ผิดโดยไม่มีใครรู้

9.4 DataFrame — หลาย Series ที่ใช้ป้ายชื่อชุดเดียวกัน

พอเข้าใจ Series แล้ว DataFrame ตรงไปตรงมามาก: **DataFrame** = ตารางที่เกิดจากหลาย **Series (หลายคอลัมน์)** มาแชร์ **index** ชุดเดียวกัน — หน้าตาเหมือน Excel sheet แต่อยู่ใน Python:

```
data = { "BTC": [63000, 65000, 64500, 66000, 65800], "ETH": [3100, 3250,
3200, 3300, 3280], "SOL": [140, 148, 145, 152, 149], } dates =
pd.date_range("2024-01-01", periods=5) # สร้างป้ายวันที่ 5 วัน df =
pd.DataFrame(data, index=dates) print(df)
```

```
► OUTPUT BTC ETH SOL 2024-01-01 63000 3100 140 2024-01-02 65000 3250 148
2024-01-03 64500 3200 145 2024-01-04 66000 3300 152 2024-01-05 65800
3280 149
```

กายวิภาคของ DataFrame มี 3 ส่วน — รู้ 3 ची้นี้แล้วอ่านเอกสาร pandas ออกทั้งหมด:

ส่วน	คืออะไร	ในตัวอย่างบน
index	ป้ายชื่อแถว (แนวตั้งซ้ายสุด)	วันที่ 2024-01-01 ถึง 05
columns	ป้ายชื่อคอลัมน์ (แนวนอนบนสุด)	BTC, ETH, SOL
values	ตัวข้อมูลข้างใน (NumPy array)	ตัวเลขราคาทั้งหมด

แล้วใช้ Excel แทนได้ไหม?

ได้ — สำหรับ "ดูข้อมูล" Excel ดีมาก แต่สำหรับงาน Quant มี 3 จุดที่ DataFrame ชนะขาด:

1. **ทำซ้ำได้:** โค้ด 10 บรรทัดรันกับข้อมูลใหม่ทุกวันอัตโนมัติ — Excel ต้องลากสูตรใหม่ทุกครั้ง
2. **ขนาด:** ข้อมูล 1 นาทีย้อนหลัง 3 ปี $\approx$ 1.5 ล้านแถว — Excel เริ่มค้าง pandas ทำงานเป็นวินาที
3. **ตรวจสอบย้อนหลังได้:** ทุกขั้นตอนคำนวณเขียนเป็นโค้ดอ่านได้ ส่งให้คนอื่น (หรือ AI) ตรวจสอบได้ — สูตรที่ฝังใน cell ของ Excel ตรวจสอบยากมาก

คิดง่าย ๆ: **Excel** = ทำกับข้าวด้วยมือ, **DataFrame** = โรงงานอาหาร — ทำกินเองมือเดียวใช้มือได้ แต่ถ้าต้องทำทุกวัน วันละหมื่นจาน ต้องใช้โรงงาน

"แล้วมันทำอะไรได้บ้าง?" — ตัวอย่างงานที่ใช้บ่อยที่สุดใน Quant แต่ละงานใช้โค้ดบรรทัดเดียว (ทุกอันได้ผลถูกต้องแม้ข้อมูลขาดบางวัน เพราะ index จับคู่ให้):

อยากได้	โค้ด
return รายวันของทุกเหรียญ	<code>df.pct_change()</code>
ค่าเฉลี่ยเคลื่อนที่ 20 วันของ BTC	<code>df["BTC"].rolling(20).mean()</code>
เฉพาะวันที่ BTC ตกเกิน 2%	<code>df[df["BTC"].pct_change() < -0.02]</code>
correlation ระหว่างทุกคู่เหรียญ	<code>df.pct_change().corr()</code>
กราฟราคาทุกเหรียญ	<code>df.plot()</code>

9.5 Operations ที่ใช้บ่อยใน Quant

```
# --- ดูข้อมูลเบื้องต้น --- df.head(3) # 3 แถวแรก df.tail(3) # 3 แถวสุดท้าย
df.describe() # mean, std, min, max, quartile df.shape # (5, 3) = 5 แถว 3
คอลัมน์ df.dtypes # type ของแต่ละคอลัมน์ # --- เลือกข้อมูล --- df["BTC"] # คอลัมน์ BTC
(Series) df[["BTC", "ETH"]] # สองคอลัมน์ (DataFrame) df.loc["2024-01-03"]
# .loc: เลือกด้วย "ป้ายชื่อ" (label) = วันที่ df.iloc[0] # .iloc: เลือกด้วย "ตำแหน่ง"
(position) = แถวที่ 0 คือแถวแรก df.iloc[-1] # .iloc[-1] = แถวสุดท้าย (เลขลบนับจากท้าย
เหมือน list) # --- คำนวณ Return --- returns = df.pct_change() # daily return
ทุก column (แถวแรกเป็น NaN เพราะไม่มีวันก่อน) log_returns = np.log(df/df.shift(1)) #
log return: shift(1) เลื่อนข้อมูลลงหนึ่งแถว (= ราคาเมื่อวาน) # --- Rolling Statistics
--- rolling_mean = df["BTC"].rolling(window=3).mean() # ค่าเฉลี่ย 3 วันล่าสุด (2
```

```
แถวแรกเป็น NaN) rolling_std = df["BTC"].rolling(window=3).std() # std 3 วันล่าสุด
zscore = (df["BTC"] - rolling_mean) / rolling_std # z-score: ห่างจากค่าเฉลี่ยกี่ std
```

.loc vs .iloc — เลือกตัวไหน?

	<code>.loc["2024-01-03"]</code>	<code>.iloc[2]</code>
เลือกด้วย	ป้ายชื่อ (วันที่, ชื่อหุ้น)	ตำแหน่ง (0, 1, 2, -1)
ข้อดี	ชัดเจน ไม่ผิดพลาดแม้เรียงลำดับใหม่	เร็ว ใช้ได้แม้ไม่รู้ชื่อ index
ใช้เมื่อ	รู้วันที่/label ที่ต้องการ	ต้องการ "ตัวแรก" หรือ "ตัวสุดท้าย"

NaN คืออะไร? "Not a Number" = "ไม่มีข้อมูล ณ จุดนี้" — pandas ใส่อัตโนมัติเมื่อคำนวณไม่ได้ (เช่น แถวแรกของ `pct_change()` หรือหน้าตาแรกของ `rolling()`) — `dropna()` ลบแถวที่มี NaN ออก, `fillna(0)` แทน NaN ด้วยศูนย์ — pandas ส่วนใหญ่ข้าม NaN ให้อัตโนมัติ (`mean()`, `std()`) แต่บางงาน (เช่น `corr()`) ต้องการ `dropna()` ก่อน

```
# --- Filter: boolean indexing — "กรองแถวด้วยเงื่อนไข" --- btc_returns =
df["BTC"].pct_change() # แทนที่จะ loop แล้ว if เองทีละแถว pandas ทำให้ในบรรทัดเดียว:
crash_days = df[btc_returns < -0.02] # เลือกเฉพาะแถวที่ return < -2% print(f"BTC
crash days (<-2%): {len(crash_days)}") # เงื่อนไขสองข้อ: ต้องใช้ & และ ครอบแต่ละเงื่อนไขด้วย
() both_up = df[(btc_returns > 0) & (df["ETH"].pct_change() > 0)]
```

ไฟล์ `btc_daily.csv` มาจากไหน?

ดาวน์โหลดข้อมูลราคา BTC จาก **CoinGecko** (ฟรี) หรือ **Binance API**:
วิธีง่ายที่สุด: เปิด https://www.coingecko.com/en/coins/bitcoin/historical_data → Export CSV → เปลี่ยนชื่อไฟล์เป็น `btc_daily.csv`
หรือใช้ Python: `pip install yfinance` แล้วรัน `yf.download("BTC-USD", start="2023-01-01").to_csv("btc_daily.csv")`

ตัวอย่างด้านล่างใช้โค้ดจำลอง (สร้างข้อมูลด้วย numpy) เพื่อรันได้ทันทีโดยไม่ต้องดาวน์โหลด

Running Example: โหลดข้อมูลจาก CSV และคำนวณ Metrics

```
# สมมติไฟล์ btc_daily.csv มีคอลัมน์ date, open, high, low, close, volume df
= pd.read_csv("btc_daily.csv", index_col="date", parse_dates=True)
df = df.sort_index() # เรียงวันที่จากเก่าไปใหม่ close = df["close"] # หยิบคอลัมน์
```

```

ราคาปิดมาตั้งชื่อสั้น ๆ # 1) return รายวัน df["return"] = close.pct_change() #
เปอร์เซ็นต์เปลี่ยนแปลงจากวันก่อน # 2) log return prev_close = close.shift(1) #
ราคาเมื่อวาน (เลื่อนลง 1 แถว) df["log_ret"] = np.log(close / prev_close) # 3)
annualized volatility จากหน้าต่าง 20 วัน daily_vol_20 =
df["return"].rolling(20).std() # std ของ return 20 วันล่าสุด df["vol_20"]
= daily_vol_20 * np.sqrt(252) # คูณ  $\sqrt{252}$  เพื่อแปลงเป็นรายปี # 4) moving
average 50 วัน df["ma_50"] = close.rolling(50).mean() # 5) z-score:
ราควันนี้ห่างค่าเฉลี่ย 60 วัน ที่ std mean_60 = close.rolling(60).mean() # ค่าเฉลี่ย
เคลื่อนที่ 60 วัน std_60 = close.rolling(60).std() # std เคลื่อนที่ 60 วัน
df["zscore"] = (close - mean_60) / std_60 # สูตร z-score เดิมจากบทที่ 7
print(df.tail()) # ดูผลลัพธ์ 5 วันล่าสุด print(f"\nAnnualized Vol (20d):
{df['vol_20'].iloc[-1]:.1%}") print(f"Current Z-score:
{df['zscore'].iloc[-1]:.2f}")

```

AI จะยุบ 5 ขั้นนี้เหลือแบบนี้ — ฝึกอ่าน **method chain**

```
df["vol_20"] = df["close"].pct_change().rolling(20).std() * np.sqrt(252)
```

เทคนิคอ่าน chain: อ่านจากซ้ายไปขวา เหมือนสายพานโรงงาน — ข้อมูลผ่านเครื่องจักรทีละตัว จุดคือ "ส่งต่อให้"

1. `df["close"]` — เริ่มจากราคาปิด
2. `.pct_change()` — ส่งเข้าเครื่องคำนวณ return รายวัน
3. `.rolling(20)` — ส่งต่อเข้าหน้าต่างเลื่อน 20 วัน
4. `.std()` — คำนวณ std ของแต่ละหน้าต่าง
5. `* np.sqrt(252)` — สุดท้ายคูณ $\sqrt{252}$ แปลงเป็นรายปี

คือขั้นที่ 1 + 3 ของเรารวมกันในบรรทัดเดียว — chain สั้น ๆ (2-3 จุด) อ่านสบาย แต่ถ้ายาวกว่านั้นแล้วเริ่มงง แดกเป็นตัวอย่างแบบที่เราเขียนได้เสมอ ผลเหมือนกันเป๊ะ และ debug ง่ายกว่าเพราะ print ดูค่ากลางทางได้

► แบบฝึกหัด: คำนวณ **Correlation Matrix**

```
# สร้าง DataFrame ของราคา 3 สินทรัพย์ data = { "BTC": [63000, 65000, 64500, 66000, 65800, 67000, 66500], "ETH": [3100, 3250, 3200, 3300, 3280, 3350, 3320], "GOLD": [1900, 1895, 1905, 1910, 1900, 1915, 1908], } df = pd.DataFrame(data) # Correlation matrix ของ returns returns = df.pct_change().dropna() corr = returns.corr() print(corr.round(3))
```

ผลลัพธ์จะแสดง matrix 3×3 — BTC/ETH มักมี correlation สูง, GOLD มัก correlation ต่ำกับ crypto

บทที่ 10: Visualization

แก่นของบทนี้

กราฟที่ตีบอกเรื่องราวได้ภายใน 3 วินาที — ใน Quant ใช้กราฟเพื่อ debug signal, ตรวจสอบ backtest, และนำเสนอผลลัพธ์ บทนี้ใช้ matplotlib เป็นหลัก (standard, ทำได้ทุกอย่าง) และแนะนำ plotly สำหรับ interactive charts

10.1 matplotlib พื้นฐาน

```
import matplotlib.pyplot as plt import numpy as np import pandas as pd #
สร้างข้อมูลตัวอย่าง dates = pd.date_range("2024-01-01", periods=60) prices = 65000
* np.cumprod(1 + np.random.normal(0.001, 0.02, 60)) # Plot พื้นฐาน fig, ax =
plt.subplots(figsize=(10, 4)) ax.plot(dates, prices, color="#0d9488",
linewidth=1.5, label="BTC Price") ax.set_title("BTC/USDT Daily Close",
fontsize=14, fontweight="bold") ax.set_xlabel("Date") ax.set_ylabel("Price
(USD)") ax.legend() ax.grid(True, alpha=0.3) plt.tight_layout()
plt.savefig("btc_price.png", dpi=150) plt.show()
```

10.2 Multiple Subplots — Price + Return + Z-score

```
# 3 panels: ราคา, return, z-score fig, axes = plt.subplots(3, 1,
figsize=(10, 10), sharex=True) # สมมติ df มีคอลัมน์ close, return, zscore # Panel
1: ราคา + MA50 axes[0].plot(df.index, df["close"], label="Close",
color="#0d9488") axes[0].plot(df.index, df["ma_50"], label="MA50",
color="#f59e0b", linestyle="--") axes[0].set_ylabel("Price ($)")
axes[0].legend() axes[0].set_title("BTC Analysis Dashboard") # Panel 2:
Daily Return — เลือกสีแท่งที่ละวัน: เขียวถ้าบวก แดงถ้าลบ bar_colors = [] for r in
df["return"]: if r > 0: bar_colors.append("#16a34a") # เขียว else:
bar_colors.append("#dc2626") # แดง axes[1].bar(df.index, df["return"],
color=bar_colors, alpha=0.7, width=0.8) axes[1].axhline(0, color="black",
linewidth=0.5) axes[1].set_ylabel("Daily Return") # Panel 3: Z-score +
threshold lines axes[2].plot(df.index, df["zscore"], color="#7c3aed",
linewidth=1) axes[2].axhline(2.0, color="#dc2626", linestyle="--",
alpha=0.7, label="Entry ( $\pm 2\sigma$ )") axes[2].axhline(-2.0, color="#dc2626",
linestyle="--", alpha=0.7) axes[2].axhline(0, color="black", linewidth=0.5)
axes[2].set_ylabel("Z-score") axes[2].legend() plt.tight_layout()
plt.savefig("btc_dashboard.png", dpi=150) plt.show()
```

AI จะยัด loop เลือกสีเข้าไปในวงเล็บเลย — พิกอ่าน

```
axes[1].bar(df.index, df["return"],  
            color=["#16a34a" if r > 0 else "#dc2626" for r in df["return"]])
```

ตรง `color=[...]` คือ comprehension ที่มีเงื่อนไขเลือกค่า (A if เงื่อนไข else B) ฝังอยู่:

1. `for r in df["return"]` — หยิบ return ที่ละวัน (loop เดิม)
2. `"#16a34a" if r > 0 else "#dc2626"` — อ่านว่า "เอาสีเขียว ถ้า r บวก ไม่จันเอาสีแดง" (รูปแบบนี้เรียก ternary — ค่าที่ได้อยู่หน้า if)
3. ผลคือ list ของสี ส่งให้ `color=` โดยตรง ไม่ต้องตั้งชื่อตัวแปร

ความหมายเหมือน loop 6 บรรทัดของเรา — ถ้าเจอแบบนี้ในโค้ด AI แล้วงง ให้ลากมันออกมาตั้งเป็นตัวแปรข้างนอกก่อน แล้วค่อยอ่าน

10.3 Scatter Plot — Correlation

```
# สร้าง DataFrame ที่มีราคา BTC และ ETH (จำลองข้อมูล 100 วัน) import numpy as np, pandas as pd  
np.random.seed(42) dates = pd.date_range("2024-01-01", periods=100)  
df_multi = pd.DataFrame({ "BTC": 65000 * np.cumprod(1 + np.random.normal(0.001, 0.02, 100)),  
                           "ETH": 3200 * np.cumprod(1 + np.random.normal(0.001, 0.025, 100)), }, index=dates)  
# วิเคราะห์ correlation ระหว่าง BTC และ ETH returns  
btc_ret = df_multi["BTC"].pct_change().dropna() # return รายวัน ลบแถวแรกที่เป็น NaN  
eth_ret = df_multi["ETH"].pct_change().dropna()  
fig, ax = plt.subplots(figsize=(6, 6)) ax.scatter(btc_ret, eth_ret, alpha=0.4, color="#0d9488", s=20)  
# เพิ่ม regression line m, b = np.polyfit(btc_ret, eth_ret, 1) # slope, intercept  
x_line = np.linspace(btc_ret.min(), btc_ret.max(), 100)  
ax.plot(x_line, m * x_line + b, color="#dc2626", linewidth=2, label=f"β={m:.2f}")  
ax.set_xlabel("BTC Daily Return")  
ax.set_ylabel("ETH Daily Return")  
ax.set_title(f"BTC vs ETH Returns (r={btc_ret.corr(eth_ret):.2f})")  
ax.legend() plt.tight_layout() plt.show()
```

Matplotlib Cheat Sheet — สำหรับ Quant

Function	ใช้ทำ
<code>ax.plot(x, y)</code>	เส้นราคา, equity curve
<code>ax.bar(x, y)</code>	return รายวัน, volume
<code>ax.scatter(x, y)</code>	correlation, scatter plot
<code>ax.fill_between(x, y1, y2)</code>	Bollinger Bands, confidence interval
<code>ax.axhline(y)</code>	เส้น threshold (entry/exit level)
<code>ax.axvspan(x1, x2)</code>	ไฮไลต์ช่วงเวลา (crisis, drawdown)

ตัวอย่าง: Equity Curve + Drawdown

```
import matplotlib.pyplot as plt import numpy as np # จำลอง equity
curve np.random.seed(42) daily_ret = np.random.normal(0.0005, 0.015,
252) equity = 100000 * np.cumprod(1 + daily_ret) # คำนวณ drawdown
running_max = np.maximum.accumulate(equity) drawdown = (equity -
running_max) / running_max # Plot fig, (ax1, ax2) = plt.subplots(2,
1, figsize=(10, 6), sharex=True) ax1.plot(equity, color="#0d9488",
linewidth=1.5) ax1.set_ylabel("Portfolio Value ($)")
ax1.set_title("Strategy Equity Curve") ax1.grid(True, alpha=0.3)
ax2.fill_between(range(len(drawdown)), drawdown, 0, color="#dc2626",
alpha=0.5) ax2.set_ylabel("Drawdown") ax2.set_xlabel("Trading Day")
ax2.grid(True, alpha=0.3) max_dd = drawdown.min() final = equity[-1]
print(f"Final Equity: ${final:,.0f}") print(f"Total Return: {(final/
100000-1):.1%}") print(f"Max Drawdown: {max_dd:.1%}")
plt.tight_layout() plt.show()
```

10.4 plotly — Interactive Charts (แนะนำ)

```
# pip install plotly import plotly.graph_objects as go fig = go.Figure() #
Candlestick chart fig.add_trace(go.Candlestick( x=df.index,
open=df["open"], high=df["high"], low=df["low"], close=df["close"],
name="BTC/USDT" )) # เพิ่ม MA fig.add_trace(go.Scatter( x=df.index,
y=df["ma_50"], line=dict(color="#f59e0b", width=1.5), name="MA50" ))
fig.update_layout( title="BTC/USDT Candlestick", yaxis_title="Price (USD)",
```

```
axis_rangeslider_visible=False, template="plotly_dark" ) fig.show() # เปิดใน
browser – zoom, pan ได้
```

► แบบฝึกหัด: Plot Spread Z-score พร้อม Entry/Exit Signals

```
import matplotlib.pyplot as plt import numpy as np # จำลอง Z-score
np.random.seed(0) z = np.cumsum(np.random.normal(0, 0.2, 200)) # OU-
like entry = 2.0 exit_z = 0.5 fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(z, color="#0d9488", linewidth=1, label="Z-score")
ax.axhline(entry, color="#dc2626", linestyle="--", label=f"Short entry
(>{entry})") ax.axhline(-entry, color="#16a34a", linestyle="--",
label=f"Long entry (<{-entry})") ax.axhline(exit_z, color="gray",
linestyle=":", alpha=0.7) ax.axhline(-exit_z, color="gray",
linestyle=":", alpha=0.7) ax.fill_between(range(len(z)), z, 0,
where=(np.abs(z) > entry), color="#dc2626", alpha=0.2, label="Active
position") ax.set_title("Spread Z-score with Entry/Exit Signals")
ax.legend() ax.grid(True, alpha=0.3) plt.tight_layout() plt.show()
```

จบ Part I — ทักษะที่ได้

ตอนนี้คุณสามารถ:

- อ่านโค้ดออกเสียงเป็นภาษาไทยได้ที่ละบรรทัด — รวมถึง comprehension และ method chain ที่ AI ชอบเขียน ✓
- เขียน Python function, loop, และ conditional logic สำหรับ signal generation ✓
- เก็บข้อมูลด้วย list, dict, tuple, set เลือกถูกตามสถานการณ์ ✓
- โหลดข้อมูลตลาดเข้า pandas DataFrame คำนวณ return, rolling vol, z-score ✓
- plot equity curve, drawdown, scatter plot, candlestick ด้วย matplotlib/plotly ✓

ใน **Part II** เราจะนำ Python ที่รู้มาต่อเข้ากับ math จาก Part 0: OLS regression, optimization, linear programming, cointegration

⚠️ **ก่อนไป Part II — สิ่งสำคัญที่ต้องจำ**

1. **Lookahead bias:** signal ที่คำนวณจากข้อมูลวัน t ต้องถูก `.shift(1)` ก่อน execute วัน $t+1$ — Part IV จะอธิบายละเอียด แต่จำหลักนี้ไว้ตั้งแต่ตอนนี้
2. **Code ใน Part I ยังไม่ optimize:** loop แบบ manual ที่เขียน เร็วพอสำหรับเรียน แต่ช้ากว่า pandas vectorized ใน Part II ประมาณ 10–100× สำหรับข้อมูลหลายหมื่นแถว
3. ตัวเลข "ดูดี" ยังไม่น่าเชื่อ: Sharpe หรือ Return ที่คำนวณในตัวอย่าง Part I-II เป็น in-sample ทั้งหมด Part IV จะสอน Walk-Forward เพื่อยืนยันว่าดีจริงหรือ overfit